Indexing

Indexes

- In MongoDB, an index is a data structure that captures a subset of a collection's data in an easy to traverse form. MongoDB indexes use a B-tree data structure.
- Indexes store the value of a specific field or fields, and attach a reference to the original document to each index entry.
- Indexes support efficient execution of queries in MongoDB.
- Without indexes, MongoDB must scan every document in a collection to select those documents that match query statement.
- If an appropriate index exists for a query, MongoDB uses the index to limit the number of documents it must scan.
- The index stores the value of a specific field or set of fields, ordered by the value of the field.

Use Cases

- If your application is repeatedly running queries on the same fields, you can create an index on those fields to improve performance.
- For example:

Scenario1: A human resources department often needs to look up employees by employee ID. You can create an index on the employee ID field to improve query performance…….Single Field Index

- **Scenario2:** A salesperson often needs to look up client information by location. Location is stored in an embedded object with fields like state, city, and zipcode. You can create an index on the location object to improve performance for queries on that object…….Single Field Index on an embedded document

❖ When you create an index on an embedded document, only queries that specify the entire embedded document use the index. Queries on a specific field within the document do not use the index.

- **Scenario3:** A grocery store manager often needs to look up inventory items by name and quantity to determine which items are low stock. You can create a single index on both the item and quantity fields to improve query performance……. Compound Index

Default Index:

- MongoDB creates a unique index on the _id field during the creation of a collection.
- The _id index prevents clients from inserting two documents with the same value for the _id field. We cannot drop this index.

Create an Index

```
db.collection.createIndex( <key and index type specification>, <options> )
```

- A document that contains the field and value pairs where the field is the index key and the value describes the type of index for that field.
- For an ascending index on a field, specify a value of 1. For descending index, specify a value of -1.

```
db.collection.createIndex( { name: -1 } )
```

To confirm that the index was created:

- **db.collection.getIndexes()**

```
[
  { v: 2, key: { _id: 1 }, name: '_id_' },
  { v: 2, key: { name: -1 }, name: 'name_-1' }
]
```

Specify an Index Name

- When you create an index, you can give the index a custom name.
- Giving your index a name helps distinguish different indexes on your collection.
- For example, you can more easily identify the indexes used by a query in the query plan's ***explain*** results if your indexes have distinct names.
- To specify the index name, include the name option when you create the index:

```
db.<collection>.createIndex(
   { <field>: <value> },
   { name: "<indexName>" }
)
```

Important points:

- Index names must be unique.
- Creating an index with the name of an existing index returns an error.
- You can't rename an existing index. Instead, you must drop and recreate the index with a new name.

```
test> db.records.find({age:{$gt:22}}).explain("executionStats")
{
  explainVersion: '2',
  queryPlanner: {
    namespace: 'test.records',
    indexFilterSet: false,
    parsedQuery: { age: { '$gt': 22 } },
    queryHash: 'B0DFDEAC',
    planCacheKey: '24C011CB',
    maxIndexedOrSolutionsReached: false,
    maxIndexedAndSolutionsReached: false,
    maxScansToExplodeReached: false,
    winningPlan: {
      queryPlan: {
        stage: 'FETCH',
        planNodeId: 2,
        inputStage: {
          stage: 'IXSCAN',
          planNodeId: 1,
          keyPattern: { age: 1 },
          indexName: 'age_1',
          isMultiKey: false,
          multiKeyPaths: { age: [] },
          isUnique: false,
          isSparse: false,
          isPartial: false,
          indexVersion: 2,
          direction: 'forward',
          indexBounds: { age: [ '(22, inf.0]' ] }
        }
      },
      slotBasedPlan: {
        slots: '$$RESULT=s11 env: { s5 = KS(2B2CFE04), s2 = Nothing (SEARCH_META), s10 = {"age" : 1}, s1 = TimeZoneDatabase(America/Argentina/Ushuaia...Atlantic/Azores) (ti
KS(33FFFFFFFFFFFFFFFFFE04), s3 = 1771416882432 (NOW) }',
        stages: '[2] nlj inner [] [s4, s7, s8, s9, s10] \n' +
          '    left \n' +
          '        [1] cfilter {(exists(s5) && exists(s6))} \n' +
          '        [1] ixseek s5 s6 s9 s4 s7 s8 [] @"71c42a80-52c1-4a29-8f1f-b058a15086b1" @"age_1" true \n' +
          '    right \n' +
          '        [2] limit 1 \n' +
          '        [2] seek s4 s11 s12 s7 s8 s9 s10 [] @"71c42a80-52c1-4a29-8f1f-b058a15086b1" true false \n'
      }
    },
    rejectedPlans: []
  },
  executionStats: {
    executionSuccess: true,
    nReturned: 2,
    executionTimeMillis: 12,
    totalKeysExamined: 2,
```

Default Index Names

- If you don't specify a name during index creation, the system generates the name by concatenating each index key field and value with underscores. For example:

Index	Default Name
`{ score : 1 }`	`score_1`
`{ content : "text", "description.tags": "text" }`	`content_text_description.tags_text`
`{ category : 1, locale : "2dsphere"}`	`category_1_locale_2dsphere`
`{ "fieldA" : 1, "fieldB" : "hashed", "fieldC" : -1 }`	`fieldA_1_fieldB_hashed_fieldC_-1`

Drop an Index

- You can remove a specific index from a collection.
- You may need to drop an index if you see a negative performance impact, want to replace it with a new index, or no longer need the index.
- To drop an index, use one of the following shell methods:

Method	Description
`db.collection.dropIndex()`	Drops a specific index from the collection.
`db.collection.dropIndexes()`	Drops all removable indexes from the collection or an array of indexes, if specified.

drop the _id index, you must drop the entire collection.

- If you drop an index that's actively used in production, you may experience performance degradation. Before you drop an index, consider **hiding the index** to evaluate the potential impact of the drop.

```
db.collection.hideIndex({age:1})
db.collection.unhideIndex({age:1})
```

Procedure:

- To drop an index, you need its name. To get all index names for a collection, run the getIndexes() method:

```
db.<collection>.getIndexes()
```

- After you identify which indexes to drop, use one of the following drop methods for the specified collection:

✔ **Drop a Single Index:** use the dropIndex() method and specify the index name:

```
db.<collection>.dropIndex("<indexName>")
```

✔ **Drop Multiple Indexes:** use the dropIndexes() method and specify an array of index names:

```
db.<collection>.dropIndexes( [ "<index1>", "<index2>", "<index3>" ] )
```

✔ **Drop All Indexes Except the _id Index:**

```
db.<collection>.dropIndexes()
```

Results: After you drop an index, the system returns information about the status of the operation.

```
...
{ "nIndexesWas" : 3, "ok" : 1 }
...
```

- The value of nIndexesWas reflects the number of indexes before removing an index.
- To confirm that the index was dropped, run:db.collection.getIndexes() method.
- The dropped index no longer appears in the getIndexes() output.

Index Types

1. **Single Field Indexes:** Single field indexes collect, sort and store data from a single field in each document in a collection.

- By default, all collections have an index on the _id field. You can add additional indexes to speed up important queries and operations.
- You can create a single-field index on any field in a document, including:

✔ Top-level document fields

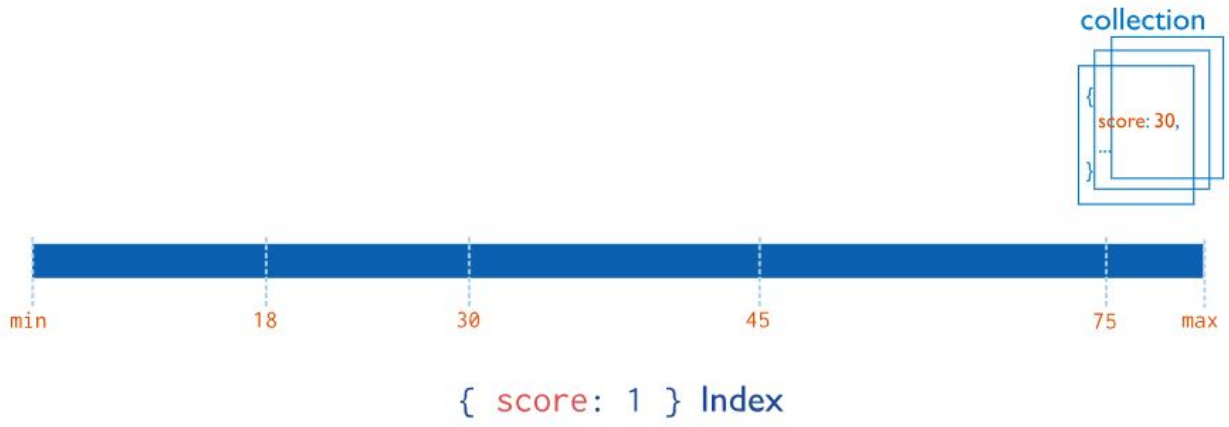
✔ Embedded documents

✔ Fields within embedded documents

- To create a single-field index, use the following prototype:

```
db.<collection>.createIndex( { <field>: <sortOrder> } )
```

- This image shows an ascending index on a single field, score:



In this example, each document in the collection that has a value for the score field is added to the index in ascending order.

Use Case:

If your application repeatedly runs queries on the same field, you can create an index on that field to improve performance. For example, your human resources department often needs to look up employees by employee ID. You can create an index on the employee ID field to improve the performance of that query.

Index Sort Order: For a single-field index, the sort order (ascending or descending) of the index key does not matter because MongoDB can traverse the index in either direction.

Example: Create a *students* collection that contains the following documents:

```
db.students.insertMany( [
   {
      "name": "Alice",
      "gpa": 3.6,
      "location": { city: "Sacramento", state: "California" }
   },
   {
      "name": "Bob",
      "gpa": 3.2,
      "location": { city: "Albany", state: "New York" }
   }
] )
```

Create an Index on a Single Field: Consider a school administrator who frequently looks up students by their GPA. You can create an index on the gpa field to improve performance for those queries:

```
db.students.createIndex( { gpa: 1 } )
```

Results: The index supports queries that select on the field gpa, such as the following:

```
db.students.find( { gpa: 3.6 } )

db.students.find( { gpa: { $lt: 3.4 } } )
```

Create an Index on an Embedded Field: You can create indexes on fields within embedded documents. Indexes on embedded fields can fulfill queries that use dot notation.

```
db.students.insertMany( [
   {
      "name": "Alice",
      "gpa": 3.6,
      "location": { city: "Sacramento", state: "California" }
   },
   {
      "name": "Bob",
      "gpa": 3.2,
      "location": { city: "Albany", state: "New York" }
   }
] )
```

- The location field is an embedded document that contains the embedded fields city and state. Create an index on the location.state field:

```
db.students.createIndex( { "location.state": 1 } )
```

Results: The index supports queries on the field location.state, such as the following:

```
db.students.find( { "location.state": "California" } )

db.students.find( { "location.city": "Albany", "location.state": "New York" } )
```

Create an Index on an Embedded Document

- You can create indexes on embedded documents as a whole.
- However, only queries that specify the **entire** embedded document use the index.
- Queries on a specific field within the document do not use the index.
- To utilize an index on an embedded document, your query must specify the entire embedded document. This can lead to unexpected behaviors if your schema model changes and you add or remove fields from your indexed document.
- When you query embedded documents, the order that you specify fields in the query matters. The embedded documents in your query and returned document must match exactly.

Example:

```
db.students.insertMany( [
   {
      "name": "Alice",
      "gpa": 3.6,
      "location": { city: "Sacramento", state: "California" }
   },
   {
      "name": "Bob",
      "gpa": 3.2,
      "location": { city: "Albany", state: "New York" }
   }
] )
```

Create an index on the location field:

```
db.students.createIndex( { location: 1 } )
```

Results: The following query uses the index on the location field:

```
db.students.find( { location: { city: "Sacramento", state: "California" } } )
```

The following queries *do not* use the index on the location field because they query on specific fields within the embedded document:

```
db.students.find( { "location.city": "Sacramento" } )
```

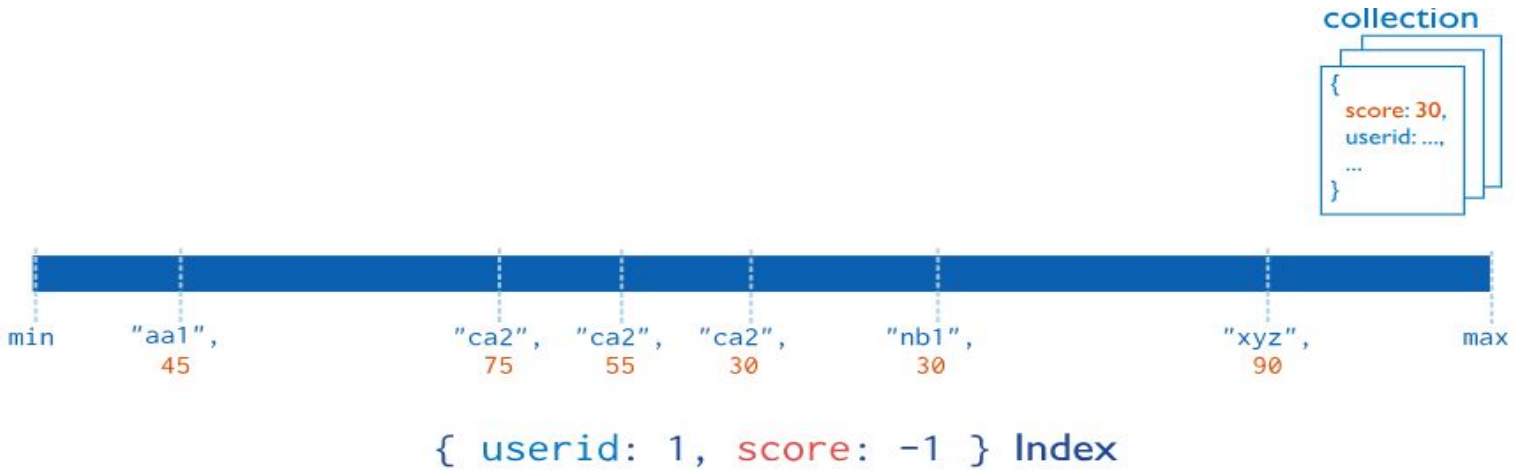
```
db.students.find( { "location.state": "New York" } )
```

- In order for a dot notation query to use an index, you must create an index on the specific embedded field you are querying, not the entire embedded object.
- Following query returns no results because the embedded fields in the query predicate are specified in a different order than they appear in the document:

```
db.students.find( { location: { state: "California", city: "Sacramento" } } )
```

2. Compound Indexes

- Compound indexes collect and sort data from two or more fields in each document in a collection.
- Data is grouped by the first field in the index and then by each subsequent field.
- For example, the following image shows a compound index where documents are first grouped by userid in ascending order (alphabetically). Then, the scores for each userid are sorted in descending order:



- Restriction: You can specify up to 32 fields in a single compound index.

- To create a compound index, use the following prototype:

```
db.<collection>.createIndex( {
   <field1>: <sortOrder>,
   <field2>: <sortOrder>,
   ...
   <fieldN>: <sortOrder>
} )
```

Use Case:

- If your application repeatedly runs a query that contains multiple fields, you can create a compound index to improve performance for that query. For example, a grocery store manager often needs to look up inventory items by name and quantity to determine which items are low stock. You can create a compound index on both the item and quantity fields to improve query performance.
- A compound index on commonly queried fields increases the chances of ***covering*** those queries. ***Covered queries*** are queries that can be satisfied entirely using an index, without examining any documents. This optimizes query performance.

Example: Create a students collection that contains these documents:

```
db.students.insertMany([
   {
      "name": "Alice",
      "gpa": 3.6,
      "location": { city: "Sacramento", state: "California" }
   },
   {
      "name": "Bob",
      "gpa": 3.2,
      "location": { city: "Albany", state: "New York" }
   }
])
```

The following operation creates a compound index containing the name and gpa fields:

```
db.students.createIndex( {
   name: 1,
   gpa: -1
} )
```

In this example:

- The index on name is ascending (1).
- The index on gpa is descending (-1).

Results: The created index supports queries that select on:

- Both name and gpa fields.
- Only the name field, because name is a prefix of the compound index.

For example, the index supports these queries:

```
db.students.find( { name: "Alice", gpa: 3.6 } )
```

```
db.students.find( { name: "Bob" } )
```

The index **does not** support queries on only the gpa field, because gpa is not part of the index prefix. For example, the index does not support this query:

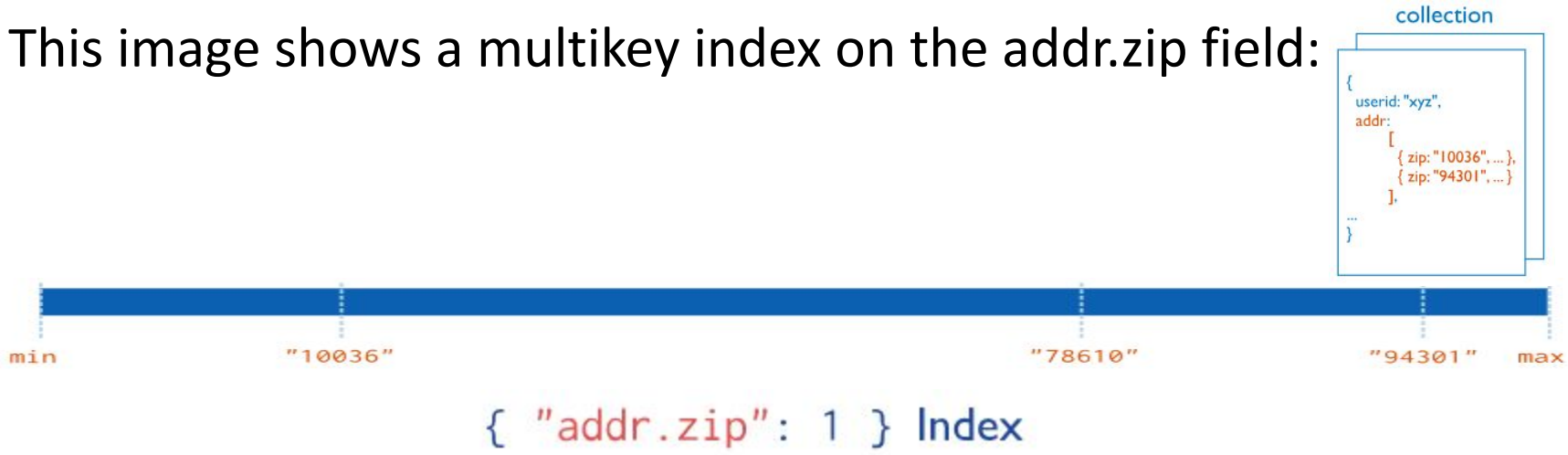
```
db.students.find( { gpa: { $gt: 3.5 } } )
```

3. Multikey Indexes

- Multikey indexes collect and sort data from fields containing array values. Improve performance for queries on array fields.
- You do not need to explicitly specify the multikey type. When you create an index on a field that contains an array value, MongoDB automatically sets that index to be a multikey index.
- MongoDB can create multikey indexes over arrays that hold both scalar values (for example, strings and numbers) and embedded documents. If an array contains multiple instances of the same value, the index only includes one entry for the value.

```
db.<collection>.createIndex( { <arrayField>: <sortOrder> } )
```

- This image shows a multikey index on the addr.zip field:



Use Cases:

- If your application frequently queries a field that contains an array value, a multikey index improves performance for those queries.
- Indexing commonly queried fields increases the chances of ***covering*** those queries. Covered queries are queries that can be satisfied entirely using an index, without examining any documents. This optimizes query performance.
- For example, documents in a students collection contain a test_scores field: an array of test scores a student received throughout the semester. You regularly update a list of top students: students who have at least five test_scores greater than 90.
- You can create an index on the test_scores field to improve performance for this query. Because test_scores contains an array value, MongoDB stores the index as a multikey index.

Create an Index on an Array Field:

- You can create an index on a field containing an array value to improve performance for queries on that field. When you create an index on a field containing an array value, MongoDB stores that index as a multikey index.

```
db.<collection>.createIndex( { <field>: <sortOrder> } )
```

```
db.students.insertMany( [
   {
      "name": "Andre Robinson",
      "test_scores": [ 88, 97 ]
   },
   {
      "name": "Wei Zhang",
      "test_scores": [ 62, 73 ]
   },
   {
      "name": "Jacob Meyer",
      "test_scores": [ 92, 89 ]
   }
] )
```

You regularly run a query that returns students with at least one test_score greater than 90. You can create an index on the test_scores field to improve performance for this query.

```
db.students.createIndex( { test_scores: 1 } )
```

This operation creates an ascending multikey index on the test_scores field of the students collection.
Because test_scores contains an array value, MongoDB stores this index as a multikey index.

Results:

- The index contains a key for each individual value that appears in the test_scores field. The index is ascending, meaning the keys are stored in this order: [62, 73, 88, 89, 92, 97].
- The index supports queries that select on the test_scores field. For example, the following query returns documents where at least one element in the test_scores array is greater than 90:

```
db.students.find(
   {
      test_scores: { $elemMatch: { $gt: 90 } }
   }
)
```

Output

```
[
   {
      _id: ObjectId("632240a20646eaee87a56a80"),
      name: 'Andre Robinson',
      test_scores: [ 88, 97 ]
   },
   {
      _id: ObjectId("632240a20646eaee87a56a82"),
      name: 'Jacob Meyer',
      test_scores: [ 92, 89 ]
   }
]
```

Compound Multikey Indexes:

- In a compound multikey index, each indexed document can have ***at most* one** indexed field whose value is an array.
- Specifically: You cannot create a compound multikey index if more than one field in the index specification is an array. For example, consider a collection that contains this document:

```
{ _id: 1, scores_spring: [ 8, 6 ], scores_fall: [ 5, 9 ] }
```

- You can't create the compound multikey index { scores_spring: 1, scores_fall: 1 } because both fields in the index are arrays.
- If a compound multikey index already exists, you cannot insert a document that would violate this restriction. Consider a collection that contains these documents:

```
{ _id: 1, scores_spring: [8, 6], scores_fall: 9 }
{ _id: 2, scores_spring: 6, scores_fall: [5, 7] }
```

- You can create a compound multikey index { scores_spring: 1, scores_fall: 1 } because for each document, only one field indexed by the compound multikey index is an array. No document contains array values for both scores_spring and scores_fall fields.
- However, after you create the compound multikey index, if you attempt to insert a document where both scores_spring and scores_fall fields are arrays, the insert fails.

Create an Index on an Embedded Field in an Array:

```
db.inventory.insertMany( [
   {
      "item": "t-shirt",
      "stock": [
         {
            "size": "small",
            "quantity": 8
         },
         {
            "size": "large",
            "quantity": 10
         },
      ]
   },
   {
      "item": "sweater",
      "stock": [
         {
            "size": "small",
            "quantity": 4
         },
         {
            "size": "large",
            "quantity": 7
         },
      ]
   },
   {
      "item": "vest",
      "stock": [
         {
            "size": "small",
            "quantity": 6
         },
         {
            "size": "large",
            "quantity": 1
         }
      ]
   }
] )
```

You need to order more inventory any time you have less than five of an item in stock. To find which items to reorder, you query for documents where an element in the stock array has a quantity less than 5. To improve performance for this query, you can create an index on the stock.quantity field.

Procedure: The following operation creates an ascending multikey index on the stock.quantity field of the inventory collection:

```
db.inventory.createIndex( { "stock.quantity": 1 } )
```

Because stock contains an array value, MongoDB stores this index as a multikey index.

Results:

- The index contains a key for each individual value that appears in the stock.quantity field. The index is ascending, meaning the keys are stored in this order: [1, 4, 6, 7, 8, 10].
- The index supports queries that select on the stock.quantity field. For example, the following query returns documents where at least one element in the stock array has a quantity less than 5:

```
db.inventory.find(
   {
      "stock.quantity": { $lt: 5 }
   }
)
```

Output

```
[
  {
    _id: ObjectId("63449793b1fac2ee2e957ef3"),
    item: 'vest',
    stock: [ { size: 'small', quantity: 6 }, { size: 'large', quantity: 1 } ]
  },
  {
    _id: ObjectId("63449793b1fac2ee2e957ef2"),
    item: 'sweater',
    stock: [ { size: 'small', quantity: 4 }, { size: 'large', quantity: 7 } ]
  }
]
```

4. Text Indexes

- Text indexes support ***text search queries*** on fields containing string content.
- Text indexes improve performance when searching for specific words or phrases within string content.
- A collection can only have **one** text index, but that index can cover multiple fields.

```
db.<collection>.createIndex(
    {
        <field1>: "text",
        <field2>: "text",
        ...
    }
)
```

Use Cases:

Documents in an online shop's clothing collection include a description field that contains a string of text describing each item. To find clothes made of silk, create a text index on the description field and run a text search query for documents with the keyword silk. The search returns all documents that mention silk in the description field.

Text Search Support:

Text indexes support ***$text*** query operations on on-premises deployments. To perform text searches, you must create a text index and use the $text query operator.

Compound Text Indexes

For a compound index that includes a text index key along with keys of other types, only the text index field determines whether the index references a document. The other keys do not determine whether the index references the documents.

```
db.blog.insertMany( [
   {
     _id: 1,
     content: "This morning I had a cup of coffee.",
     about: "beverage",
     keywords: [ "coffee" ]
   },
   {
     _id: 2,
     content: "Who likes chocolate ice cream for dessert?",
     about: "food",
     keywords: [ "poll" ]
   },
   {
     _id: 3,
     content: "My favorite flavors are strawberry and coffee",
     about: "ice cream",
     keywords: [ "food", "dessert" ]
   }
] )
```

```
[
   {
     _id: 1,
     content: 'This morning I had a cup of coffee.',
     about: 'beverage',
     keywords: [ 'coffee' ]
   },
   {
     _id: 3,
     content: 'My favorite flavors are strawberry and coffee',
     about: 'ice cream',
     keywords: [ 'food', 'dessert' ]
   }
]
```

Create a Single-Field Text Index

```
db.blog.createIndex( { "content": "text" } )
```

The index supports text search queries on the *content* field. For example, the following query returns documents where the *content* field contains the string *coffee*:

```
db.blog.find(
   {
      $text: { $search: "coffee" }
   }
)
```

The data is stored in the format of the document, it can hold a huge amount of data. So searching is an important criteria here and for that MongoDB provides Text indexes to support text search queries especially on string content.

Matches on Non-Indexed Fields:

- The *{ "content": "text" }* index only includes the *content* field, and does not return matches on non-indexed fields. For example, the following query searches the *blog* collection for the string *food*:

```
db.blog.find(
   {
      $text: { $search: "food" }
   }
)
```

- The preceding query returns no documents. Although the string food appears in documents *_id: 2* and *_id: 3*, it appears in the *about* and *keywords* fields respectively.
  The *about* and *keywords* fields are not included in the text index, and therefore do not affect text search query results.

Create a Compound Text Index

```
db.blog.insertMany( [
   {
     _id: 1,
     content: "This morning I had a cup of coffee.",
     about: "beverage",
     keywords: [ "coffee" ]
   },
   {
     _id: 2,
     content: "Who likes chocolate ice cream for dessert?",
     about: "food",
     keywords: [ "poll" ]
   },
   {
     _id: 3,
     content: "My favorite flavors are strawberry and coffee",
     about: "ice cream",
     keywords: [ "food", "dessert" ]
   }
] )
```

Create a compound text index on the *about* and *keywords* fields in the blog collection:

```
db.blog.createIndex(
   {
      "about": "text",
      "keywords": "text"
   }
)
```

The index supports text search queries on the *about* and *keywords* fields. For example, the following query returns documents where the string *food* appears in either the *about or keywords* field:

```
db.blog.find(
   {
      $text: { $search: "food" }
   }
)
```

```
[
  {
    _id: 3,
    content: 'My favorite flavors are strawberry and coffee',
    about: 'ice cream',
    keywords: [ 'food', 'dessert' ]
  },
  {
    _id: 2,
    content: 'Who likes chocolate ice cream for dessert?',
    about: 'food',
    keywords: [ 'poll' ]
  }
```

$text

- $text performs a text search on the content of the fields indexed with a text index.

```
{
  $text: {
    $search: <string>,
    $language: <string>,
    $caseSensitive: <boolean>,
    $diacriticSensitive: <boolean>
  }
}
```

- **$search:** In the $search field, specify a string of words that the $text operator parses and uses to query the text index.
- **$caseSensitive:** A boolean flag to enable or disable case sensitive search. **Defaults to false**; i.e. the search defers to the case insensitivity of the text index.
- **$diacriticSensitive:** A boolean flag to enable or disable diacritic sensitive search. **Defaults to false**; i.e. the search defers to the diacritic insensitivity of the text index.

Example:

```
db.articles.insertMany( [
     { _id: 1, subject: "coffee", author: "xyz", views: 50 },
     { _id: 2, subject: "Coffee Shopping", author: "efg", views: 5 },
     { _id: 3, subject: "Baking a cake", author: "abc", views: 90  },
     { _id: 4, subject: "baking", author: "xyz", views: 100 },
     { _id: 5, subject: "Café Con Leche", author: "abc", views: 200 },
     { _id: 6, subject: "Сырники", author: "jkl", views: 80 },
     { _id: 7, subject: "coffee and cream", author: "efg", views: 10 },
     { _id: 8, subject: "Cafe con Leche", author: "xyz", views: 10 }
] )
```

```
db.articles.createIndex( { subject: "text" } )
```

Search for a Single Word

```
db.articles.find( { $text: { $search: "coffee" } } )
```

This query returns the documents that contain the term coffee in the indexed subject field, or more precisely, the stemmed version of the word:

```
{ _id: 1, subject: 'coffee', author: 'xyz', views: 50 },
{ _id: 7, subject: 'coffee and cream', author: 'efg', views: 10 },
{ _id: 2, subject: 'Coffee Shopping', author: 'efg', views: 5 }
```

Example:

```
db.articles.insertMany( [
     { _id: 1, subject: "coffee", author: "xyz", views: 50 },
     { _id: 2, subject: "Coffee Shopping", author: "efg", views: 5 },
     { _id: 3, subject: "Baking a cake", author: "abc", views: 90  },
     { _id: 4, subject: "baking", author: "xyz", views: 100 },
     { _id: 5, subject: "Café Con Leche", author: "abc", views: 200 },
     { _id: 6, subject: "Сырники", author: "jkl", views: 80 },
     { _id: 7, subject: "coffee and cream", author: "efg", views: 10 },
     { _id: 8, subject: "Cafe con Leche", author: "xyz", views: 10 }
] )
```

```
db.articles.createIndex( { subject: "text" } )
```

Searching for an exact multi-word phrase:

If you search for Coffee Shopping, MongoDB normally finds documents containing either “Coffee" OR “Shopping". To find only the exact phrase “Coffee Shopping," use:

db.stores.find({ $text: { $search: "\“Coffee Shopping\"" } }).

Specify the Default Language for a Text Index:

- By default, the default_language for text indexes is english. To improve the performance of non-English text search queries, you can specify a different default language associated with your text index.
- The default language associated with the indexed data determines the suffix stemming rules. The default language also determines which language-specific stop words (for example, the, an, a, and and in English) are not indexed.
- To specify a different language, use the default_language option when creating the text index.

```
db.<collection>.createIndex(
   { <field>: "text" },
   { default_language: <language> }
)
```

- Stop words are **language-specific** (MongoDB uses language rules like English, etc.) They are removed during **index creation** and during **search queries**

Stop words usage example

- Suppose you have a document:

  { text: "The AI is in the lab" }

- When MongoDB creates a **text index**, it processes the sentence:

  Original words: The, AI, is, in, the, lab

  After removing stop words: AI, lab

- Only **“AI”** and **“lab”** are stored in the index and words like **“the”, “is”, “in”** are completely ignored
- **Impact on search**
- **Query 1:**

  db.collection.find({ $text: { $search: "AI" } })

  Works correctly (AI is indexed)

- **Query 2:**

  db.collection.find({ $text: { $search: "the" } })

  Returns nothing (because “the” is not indexed)

5. Hashed Indexes

- A **hashed index** in MongoDB is an index where the **value of a field is converted into a hash**, and that hash is what gets stored and indexed.
- Instead of indexing actual values e.g. { userId: 12345 }, MongoDB stores something like: { userId: hash(12345) }
- So the index is built on the **hashed value**, not the original value.
- Hashed indexes are ideal for equality queries where they help to retrieve results through fast matches.
- E.g. db.users.find({ userId: 12345 })
- **With hashed index range queries don't work efficiently**

  db.users.find({ userId: { $gt: 1000 } })
- Hash values have **no order**, so MongoDB cannot perform range scans
- Also, sorting is **not** supported through hashed indexes.

  E.g. db.users.find().sort({ userId: 1 })

Hashed Indexes

- *Hashed indexes support sharding using hashed shard keys. Hashed based sharding uses a hashed index of a field as the shard key to partition data across your sharded cluster.*

Hashing Function: When MongoDB uses a hashed index to resolve a query, it uses a hashing function to automatically compute the hash values. Applications do **not** need to compute hashes.

Embedded Documents:

- MongoDB **does NOT allow hashed indexes directly on an entire embedded document (object)** because hashing requires a single scalar value, not a full object.
- You can create a hashed index on a **specific field inside the embedded document**.
- The hashing function collapses embedded documents and computes the hash for the entire value.

Floating-Point Numbers

- Hashed indexes truncate floating-point numbers to 64-bit integers before hashing. For example, a hashed index uses the same hash to store the values 2.3, 2.2, and 2.9. This is a **collision**, where multiple values are assigned to a single hash key. Collisions may negatively impact query performance.
- To prevent collisions, do not use a hashed index for floating-point numbers that cannot be reliably converted to 64-bit integers and then back to floating point.
- Hashed indexes do not support floating-point numbers larger than 2^{53}

Limitations

- Hashed indexes have limitations for array fields and the unique property.

Array Fields

- The hashing function does not support multi-key indexes.
- You cannot create a hashed index on a field that contains an array *or* insert an array into a hashed indexed field.

Unique Constraint

- You cannot specify a unique constraint on a hashed index i.e. {unique: true} is not allowed.
- Instead, you can create an additional non-hashed index with the unique constraint. MongoDB can use that non-hashed index to enforce uniqueness on the chosen field.

```
// Enforces that every user has a unique username
db.users.createIndex({ username: 1 }, { unique: true });
```

Create a Hashed Index:

- To create a single-field hashed index, specify hashed as the value of the index key:

```
db.<collection>.createIndex(
   {
      <field>: "hashed"
   }
)
```

- To create a hashed index that contains multiple fields (a compound hashed index), specify hashed as the value of a *single* index key. For other index keys, specify the sort order (1 or -1):

```
db.<collection>.createIndex(
   {
      <field1>: "hashed",
      <field2>: "<sort order>",
      <field3>: "<sort order>",
      ...
   }
)
```

- Hashed index can contain either a single field or multiple fields. The fields in your index specify how data is distributed across shards in your cluster.

Create a Single-Field Hashed Index

- Consider an orders collection that already contains data. Create a hashed index in the orders collection on the _id field:

```
db.orders.createIndex( { _id: "hashed" } )
```

- The _id field increases monotonically, which makes it a good candidate for a hashed index key. Although _id values incrementally increase, when MongoDB generates a hash for individual _id values, those hashed values are unlikely to be on the same chunk.

Create a Compound Hashed Index

- Consider a customers collection that already contains data. Create a compound hashed index in the customers collection on the name, address, and birthday fields:

```
db.customers.createIndex(
   {
      "name" : 1
      "address" : "hashed",
      "birthday" : -1
   }
)
```

- When you create a compound hashed index, you **must specify hashed as the value of a *single* index key**. For other index keys, specify the sort order (1 or -1). In the preceding index, address is the hashed field.

6. Geospatial Indexes

- It helps to find location near another location.
- Geospatial indexes support queries on data stored as **GeoJSON objects** or **legacy coordinate pairs**. You can use geospatial indexes to improve performance for queries on geospatial data or to run certain geospatial queries.
- MongoDB provides two types of geospatial indexes:

✔ 2dsphere Indexes, which support queries that interpret geometry on a sphere.

✔ 2d Indexes, which support queries that interpret geometry on a flat surface.

```
{
Name: “TU”
Location: { type: “Point”, coordinates: [-80.22,40.76]}
Category: “University”
}
{
Name: “PU”
Location: { type: “Point”, coordinates: [-90.22,70.76]}
Category: “University”
}
```

GeoJSON Objects

- To calculate geometry over an Earth-like sphere, store your location data as GeoJSON objects.

 To specify GeoJSON data, use an embedded document with:

✔ a field named type that specifies the GeoJSON object type and

✔ a field named coordinates that specifies the object's coordinates.

- If specifying latitude and longitude coordinates, list the **longitude** first, and then **latitude**.
- Valid longitude values are between -180 and 180, both inclusive.
- Valid latitude values are between -90 and 90, both inclusive.

```
<field>: { type: <GeoJSON type> , coordinates: <coordinates> }
```

- For example, to specify a GeoJSON Point:

```
location: {
       type: "Point",
       coordinates: [-73.856077, 40.848447]
}
```

- MultiPoint: "coordinates" member is an array of positions.

```
{
  type: "MultiPoint",
  coordinates: [
     [ -73.9580, 40.8003 ],
     [ -73.9498, 40.7968 ],
     [ -73.9737, 40.7648 ],
     [ -73.9814, 40.7681 ]
  ]
}
```

Point: the "coordinates" member is a single position.

```
{ type: "Point", coordinates: [ 40, 5 ] }
```

MultiPoint: "coordinates" member is an array of positions.

```
{
  type: "MultiPoint",
  coordinates: [
     [ -73.9580, 40.8003 ],
     [ -73.9498, 40.7968 ],
     [ -73.9737, 40.7648 ],
     [ -73.9814, 40.7681 ]
  ]
}
```

LineString: For type "LineString", the "coordinates" member is an array of two or more positions.

```
{ type: "LineString", coordinates: [ [ 40, 5 ], [ 41, 6 ] ] }
```

MultiLineString For type "MultiLineString", the "coordinates" member is an array of LineString coordinate arrays.

```
{
  type: "MultiLineString",
  coordinates: [
     [ [ -73.96943, 40.78519 ], [ -73.96082, 40.78095 ] ],
     [ [ -73.96415, 40.79229 ], [ -73.95544, 40.78854 ] ],
     [ [ -73.97162, 40.78205 ], [ -73.96374, 40.77715 ] ],
     [ [ -73.97880, 40.77247 ], [ -73.97036, 40.76811 ] ]
  ]
}
```

Polygon

Multipolygon

```
{
  type : "Polygon",
  coordinates : [
     [ [ 0 , 0 ] , [ 3 , 6 ] , [ 6 , 1 ] , [ 0 , 0 ] ],
     [ [ 2 , 2 ] , [ 3 , 3 ] , [ 4 , 2 ] , [ 2 , 2 ] ]
  ]
}
```

Legacy Coordinate Pairs:

- To calculate distances on a Euclidean plane, store your location data as legacy coordinate pairs and use a 2d index.
- MongoDB supports spherical surface calculations on legacy coordinate pairs by using a 2dsphere index if you manually convert the data to the GeoJSON Point type.
- To specify data as legacy coordinate pairs, you can use either an array (*preferred*) or an embedded document.
- ***Specify via an array:*** `<field>: [ <x>, <y> ]`
- If ***specifying latitude and longitude coordinates***, list the **longitude** first and then **latitude**; i.e.

```
<field>: [<longitude>, <latitude> ]
```

- Valid longitude values are between -180 and 180, both inclusive.
- Valid latitude values are between -90 and 90, both inclusive.

- ***Specify via an embedded document:***

```
<field>: { <field1>: <x>, <field2>: <y> }
```

- If specifying latitude and longitude coordinates, the first field, regardless of the field name, must contain the **longitude** value and the second field, the **latitude** value ; i.e.

```
<field>: { <field1>: <longitude>, <field2>: <latitude> }
```

- Valid longitude values are between -180 and 180, both inclusive.
- Valid latitude values are between -90 and 90, both inclusive.
- To specify legacy coordinate pairs, arrays are preferred over an embedded document as some languages do not guarantee associative map ordering.

Use Cases:

- If your application frequently queries a field that contains geospatial data, you can create a geospatial index to improve performance for those queries.
- Certain query operations require a geospatial index. If you want to query with the **$near** or **$nearSphere** operators or the **$geoNear** aggregation stage, you must create a geospatial index.
- For example, consider a subway collection with documents containing a location field, which specifies the coordinates of subway stations in a city. You often run queries with the **$geoWithin** operator to return a list of stations within a specific area. To improve performance for this query, you can create a geospatial index on the location field. After creating the index, you can query using the **$near** operator to return a list of nearby stations, sorted from nearest to farthest.
- Indexing commonly queried fields increases the chances of **covering** those queries. Covered queries are queries that can be satisfied entirely using an index, without examining any documents. This optimizes query performance.

2dsphere Indexes:

- 2dsphere indexes support geospatial queries on an earth-like sphere. For example, 2dsphere indexes can:

- Determine points within a specified area.
- Calculate proximity to a specified point.
- Return exact matches on coordinate queries.

- To create a 2dsphere index, specify the string 2dsphere as the index type:

```
db.<collection>.createIndex( { <location field> : "2dsphere" } )
```

Use Cases:

- Use 2dsphere indexes to query and perform calculations on location data where the data points appear on Earth, or another spherical surface.
- For example: A food delivery application uses 2dsphere indexes to support searches for nearby restaurants.
- A route planning application uses 2dsphere indexes to calculate the shortest distance between rest stops.
- A city planner uses 2dsphere indexes to find parks that exist within city limits.

Create a 2dsphere Index: Create a places collection that contains these documents:

```
db.places.insertMany( [
   {
      loc: { type: "Point", coordinates: [ -73.97, 40.77 ] },
      name: "Central Park",
      category : "Park"
   },
   {
      loc: { type: "Point", coordinates: [ -73.88, 40.78 ] },
      name: "La Guardia Airport",
      category: "Airport"
   },
   {
      loc: { type: "Point", coordinates: [ -1.83, 51.18 ] },
      name: "Stonehenge",
      category : "Monument"
   }
] )
```

- The values in the loc field are GeoJSON points.

Procedure: The following operation creates a 2dsphere index on the location field loc:

```
db.places.createIndex( { loc : "2dsphere" } )
```

Query for Locations Near a Point on a Sphere

- To query for location data near a specified point, use the **$near** operator:

```
db.<collection>.find( {
   <location field> : {
      $near : {
         $geometry : {
            type : "Point",
            coordinates : [ <longitude>, <latitude> ]
         },
         $maxDistance : <distance in meters>
      }
   }
} )
```

- When you specify longitude and latitude coordinates, list the **longitude** first, and then **latitude**.
  - Valid longitude values are between -180 and 180, both inclusive.
  - Valid latitude values are between -90 and 90, both inclusive.
- Specify distance in the $maxDistance field in **meters**.

- Use $near to query the collection. The following $near query returns documents that have a loc field within 5000 meters of a GeoJSON point located at [-73.92, 40.78]:

```
db.places.find( {
   loc: {
      $near: {
         $geometry: {
            type: "Point",
            coordinates: [ -73.92, 40.78 ]
         },
         $maxDistance : 5000
      }
   }
} )
```

Results are sorted by distance from the queried point, from nearest to farthest.

```
[
  {
    _id: ObjectId("63f7c3b15e5eefbdfef81cab"),
    loc: { type: 'Point', coordinates: [ -73.88, 40.78 ] },
    name: 'La Guardia Airport',
    category: 'Airport'
  },
  {
    _id: ObjectId("63f7c3b15e5eefbdfef81caa"),
    loc: { type: 'Point', coordinates: [ -73.97, 40.77 ] },
    name: 'Central Park',
    category: 'Park'
  }
]
```

```
db.places.insertMany( [
   {
      loc: { type: "Point", coordinates: [ -73.97, 40.77 ] },
      name: "Central Park",
      category : "Park"
   },
   {
      loc: { type: "Point", coordinates: [ -73.88, 40.78 ] }
      name: "La Guardia Airport",
      category: "Airport"
   },
   {
      loc: { type: "Point", coordinates: [ -1.83, 51.18 ] },
      name: "Stonehenge",
      category : "Monument"
   }
] )
```

```
db.places.createIndex( { loc : "2dsphere" } )
```

```
db.places.find( {
   loc: {
      $near: {
         $geometry: {
            type: "Point",
            coordinates: [ -73.92, 40.78 ]
         },
         $maxDistance : 5000
      }
   }
} )
```

```
[
  {
    _id: ObjectId("63f7c3b15e5eefbdfef81cab"),
    loc: { type: 'Point', coordinates: [ -73.88, 40.78 ] },
    name: 'La Guardia Airport',
    category: 'Airport'
  },
  {
    _id: ObjectId("63f7c3b15e5eefbdfef81caa"),
    loc: { type: 'Point', coordinates: [ -73.97, 40.77 ] },
    name: 'Central Park',
    category: 'Park'
  }
]
```

Covered Query

- A covered query is a query that can be satisfied entirely using an index and does not have to examine any documents. An index covers a query when all of the following apply:

- all the fields in the query are part of an index, **and**
- all the fields returned in the results are in the same index.
- no fields in the query are equal to null (i.e. {"field" : null} or {"field" : {$eq : null}}).

- For example, a collection inventory has the following index on the type and item fields:

```
db.inventory.createIndex( { type: 1, item: 1 } )
```

- This index will cover the following operation which queries on the type and item fields and returns only the item field:

```
db.inventory.find(
   { type: "food", item:/^c/ },
   { item: 1, _id: 0 }
)
```

- For the specified index to cover the query, the projection document must explicitly specify _id: 0 to exclude the _id field from the result since the index does not include the _id field.

Embedded Documents:

- An index can cover a query on fields within embedded documents. For example, consider a collection userdata with documents of the following form:

```
{ _id: 1, user: { login: "tester" } }
```

- The collection has the following index: `{ "user.login": 1 }`
- The { "user.login": 1 } index will cover the query below:

```
db.userdata.find( { "user.login": "tester" }, { "user.login": 1, _id: 0 } )
```

Performance:

- Because the index contains all fields required by the query, MongoDB can both match the query conditions and return the results using only the index.
- Querying *only* the index can be much faster than querying documents outside of the index. Index keys are typically smaller than the documents they catalog, and indexes are typically available in RAM or located sequentially on disk.

Limitations:

Restrictions on Indexed Fields

- Geospatial indexes can't cover a query.
- Multikey indexes cannot cover queries over array fields.

Partial Indexes/filtering

- Partial indexes only index the documents in a collection that meet a specified filter expression. Partial indexes have lower storage requirements and reduced performance costs for index creation and maintenance.

Create a Partial Index:

- To create a partial index, use ***db.collection.createIndex()*** method with the ***partialFilterExpression*** option.
- The ***partialFilterExpression*** option accepts a document that specifies the filter condition using:
    - equality expressions (i.e. field: value or using the **$eq** operator),
    - **$exists:** true expression,
    - **$gt, $gte, $lt, $lte** expressions,
    - **$type** expressions,
    - **$and** operator,
    - **$or** operator,
    - **$in** operator

- For example, the following operation creates a compound index that indexes only the documents with a rating field greater than 5.

```
db.restaurants.createIndex(
   { cuisine: 1, name: 1 },
   { partialFilterExpression: { rating: { $gt: 5 } } }
)
```

- You can specify a ***partialFilterExpression*** option for all MongoDB index types.

- For example, given the following index:

```
db.restaurants.createIndex(
   { cuisine: 1 },
   { partialFilterExpression: { rating: { $gt: 5 } } }
)
```

- The following query can use the index since the query predicate includes the condition rating: { $gte: 8 } that matches a subset of documents matched by the index filter expression rating: { $gt: 5 }:

```
db.restaurants.find( { cuisine: "Italian", rating: { $gte: 8 } } )
```

- However, the following query cannot use the partial index on the cuisine field because using the index results in an incomplete result set. Specifically, the query predicate includes the condition rating: { $lt: 8 } while the index has the filter rating: { $gt: 5 }. That is, the query { cuisine: "Italian", rating: { $lt: 8 } } matches more documents (e.g. an Italian restaurant with a rating equal to 1) than are indexed.

```
db.restaurants.find( { cuisine: "Italian", rating: { $lt: 8 } } )
```

- Similarly, the following query cannot use the partial index because the query predicate does not include the filter expression and using the index would return an incomplete result set.

```
db.restaurants.find( { cuisine: "Italian" } )
```